

Active Directory Windows Automation: Execution Environment Framework

Execution environment specifications for automating Active Directory-joined Windows endpoints using Ansible Automation Platform (AAP 2) via native Kerberos over WinRM HTTPS (Port 5986).

Deployment Sequence

Modules must be executed sequentially. Infrastructure prerequisites and container build steps must be locked before configuring assets inside AAP.

1. Prerequisites: Infrastructure & Team Alignment

- **Scope:** Cross-team technical alignment, network firewall requirements (Ports 88, 464, 389/636, 5986, 53), Active Directory prerequisites, NTP clock sync rules, and enterprise PKI/CRL trust configurations.

2. Execution Environment Build Guide

- **Scope:** Compiling a custom Execution Environment container image with ansible-builder, system Kerberos binaries (krb5-workstation), Python dependencies (pywinrm[kerberos]), krb5.conf formatting rules, and container registry publishing.

3. Inventory & Playbooks Guide

- **Scope:** Structuring inventory connection variables for Kerberos authentication (ansible_winrm_transport: kerberos), target host domain prerequisites, host FQDN mandates, and ready-to-use sample playbooks.

4. AAP 2 UI Integration & Execution Guide

- **Scope:** Integration inside the AAP 2 Controller web console (Execution Environments, Projects, Inventories, Credentials), Job Templates, interactive job execution, and automated kinit ticket acquisition.

5. Disconnected Supply Chain & Air-Gapped Build Guide

- **Scope:** Offline supply chain management, base image mirroring using skopeo, offline Python wheel bundling for pywinrm[kerberos], local Ansible Collection packaging, and offline RHEL RPM repos.

1. Execution Environment Build Guide

Guide for building a custom Execution Environment (EE) container image to automate Active Directory-joined Windows endpoints using Kerberos over WinRM HTTPS.

1.1 Prerequisites

- RHEL 9 / Fedora build host with ansible-builder (v3+) and podman installed.
- Access to an enterprise container registry (Quay, Harbor, Nexus, or registry.redhat.io).
- Active Directory domain details (Domain Realm Name in ALL CAPS and Domain Controller FQDNs).
- Base64 PEM-encoded (.crt/.pem) Root and Intermediate CA certificates.

1.2 Container Build Files

Create a dedicated build directory (e.g., ee-build/) containing the following build definitions.

Step 1: Kerberos Configuration (krb5.conf)

```
[libdefaults]
    default_realm = YOURDOMAIN.COM
    dns_lookup_realm = false
    dns_lookup_kdc = true
    ticket_lifetime = 24h
    renew_lifetime = 7d
    forwardable = true
    rdns = false
    pkinit_anchors = FILE:/etc/pki/tls/certs/ca-bundle.crt
    default_ccache_name = KEYRING:persistent:%{uid}
```

```
[realms]
```

```
YOURDOMAIN.COM = {  
    kdc = dc01.yourdomain.com  
    kdc = dc02.yourdomain.com  
    admin_server = dc01.yourdomain.com  
    default_domain = yourdomain.com  
}
```

```
[domain_realm]  
    .yourdomain.com = YOURDOMAIN.COM  
    yourdomain.com = YOURDOMAIN.COM
```

Kerberos Protocol Standard: Realm names (YOURDOMAIN.COM) must be specified in **ALL CAPS** across all configuration blocks. Per Kerberos RFC specifications, domain realms are case-sensitive; lowercased entries will cause ticket request failures.

Step 2: Build Definition (execution-environment.yml)

```
---  
version: 3  
  
build_arg_defaults:  
    EE_BASE_IMAGE:  
        'registry.redhat.io/ansible-automation-platform-24/ee-supported-rhel9:latest'  
  
dependencies:  
    ansible_core:  
        package_pip: ansible-core  
    system:  
        - krb5-workstation  
        - krb5-libs  
        - ca-certificates  
    python:  
        - pywinrm[kerberos]  
  
additional_build_steps:  
    prepend_base:  
        - COPY internal-root-ca.crt /etc/pki/ca-trust/source/anchors/internal-root-ca.crt  
        - RUN update-ca-trust  
    append_final:
```

```
- COPY krb5.conf /etc/krb5.conf
```

TLS Certificate Requirement: The internal Root CA (internal-root-ca.crt) must be placed in the build folder prior to compilation. Omitting internal issuing CAs causes `ssl.SSLCertVerificationError` when certificate validation is enforced over WinRM HTTPS.

Step 3: Build & Publish Container Image

1. Build the container image

```
ansible-builder build --tag my-registry.com/my-project/windows-kerberos-ee:latest
```

2. Authenticate to registry

```
podman login my-registry.com
```

3. Push image to registry

```
podman push my-registry.com/my-project/windows-kerberos-ee:latest
```

1.3 Disconnected & Air-Gapped Build Adjustments

- **Base Image:** Mirror `registry.redhat.io/ansible-automation-platform-24/ee-supported-rhel9:latest` to your private registry.
- **Python Dependencies:** Direct pip to an internal PyPI mirror (Nexus/Artifactory) via `pip.conf` or utilize pre-packaged wheel files.
- **System Packages:** Ensure `krb5-workstation` and `krb5-libs` are available via internal Red Hat Satellite or local RPM repositories.

1.4 Pre-Flight Container Validation

1. Run interactive container shell

```
podman run --rm -it my-registry.com/my-project/windows-kerberos-ee:latest /bin/bash
```

2. Confirm Kerberos binary installation

```
kinit --version
```

3. Test ticket acquisition against Active Directory

```
kinit ansible_svc@YOURDOMAIN.COM
```

```
# 4. Inspect active Kerberos ticket cache  
klist
```

```
# 5. Verify pywinrm Kerberos module loading  
python3 -c "import winrm; print(winrm)"
```

2. Inventory & Playbook Standards

Standardized inventory connection variables and verification playbooks for managing Active Directory-joined Windows endpoints via Kerberos over WinRM HTTPS (Port 5986).

2.1 WinRM Connection Variables

Configure baseline connection settings inside `group_vars/windows.yml` to enforce encrypted Kerberos transport and TLS certificate validation.

```
---  
# Connection Protocol & Port  
ansible_connection: winrm  
ansible_port: 5986  
  
# Authentication & Encryption  
ansible_winrm_transport: kerberos  
ansible_winrm_realm: YOURDOMAIN.COM  
ansible_winrm_server_cert_validation: validate  
  
# Service Account Principal (Omit domain prefix when using Kerberos)  
ansible_user: ansible_svc
```

Mandatory Host Address Standard: All inventory host definitions must use target Fully Qualified Domain Names (e.g., `win01.yourdomain.com`). Kerberos authentication resolves Service Principal Names (SPNs) bound strictly to DNS hostnames; specifying IP addresses causes authentication failures.

2.2 Inventory Definition

Example YAML inventory structure (hosts.yml):

```
---
all:
  children:
    windows:
      hosts:
        dc01.yourdomain.com:
        app01.yourdomain.com:
        db01.yourdomain.com:
      vars:
        ansible_connection: winrm
        ansible_port: 5986
        ansible_winrm_transport: kerberos
        ansible_winrm_realm: YOURDOMAIN.COM
        ansible_winrm_server_cert_validation: validate
```

2.3 Baseline Verification Playbooks

Execute these standard playbooks sequentially to validate WinRM HTTPS connectivity, Kerberos ticket delegation, and Active Directory module integration.

Playbook 1: WinRM Connectivity Check (01_winrm_ping.yml)

```
---
- name: Verify Windows WinRM Connectivity
  hosts: windows
  gather_facts: false
  tasks:
    - name: Execute WinRM ping
      ansible.windows.win_ping:

    - name: Query executing identity
      ansible.windows.win_whoami:
      register: whoami_out
```

```
- name: Display authenticated identity
  ansible.builtin.debug:
    var: whoami_out.account.sid
```

Playbook 2: Active Directory Information Retrieval (02_ad_info.yml)

```
---
- name: Query Active Directory Infrastructure
  hosts: windows
  gather_facts: false
  tasks:
    - name: Retrieve Domain Information
      microsoft.ad.domain_info:
        register: ad_info

    - name: Display Active Directory Domain Details
      ansible.builtin.debug:
        msg:
          - "Domain Name: {{ ad_info.domain.name }}"
          - "Domain Functional Level: {{ ad_info.domain.domain_mode }}"
          - "Primary Forest: {{ ad_info.domain.forest }}"
```

2.4 Common Execution Errors & Root Causes

- **HTTP 401 Unauthorized:** Kerberos realm mismatch or lowercase domain name in configuration files. Ensure realm is **ALL CAPS** (YOURDOMAIN.COM).
- **Cannot find SPN:** Target endpoint defined via IP address or missing WinRM SPN registration in Active Directory.
- **ssl.SSLCertVerificationError:** Target host TLS certificate issued by an untrusted internal CA. Ensure issuing CAs are embedded into the container trust store.
- **Kerberos clock skew too great:** Time difference between execution node, Domain Controller, and target endpoint exceeds 5 minutes.

3. AAP 2 Controller Integration &

Execution Guide

Configuration standards for registering custom Execution Environments, credentials, projects, and job templates in Ansible Automation Platform (AAP 2) Controller for Active Directory automation.

3.1 Asset Provisioning Sequence

To prevent execution failures, assets must be created in the Controller UI in this strict order:

1. Execution Environment → 2. Credentials → 3. Project → 4. Inventory → 5. Job Template

3.2 Controller Asset Configuration

Step 1: Register Execution Environment

Navigate to **Execution Environments > Add:**

- **Name:** Windows Active Directory EE
- **Image:** my-registry.com/my-project/windows-kerberos-ee:latest
- **Pull:** Always (or Only if not present based on registry access)
- **Organization:** Select designated organization

Step 2: Configure Machine Credentials

Navigate to **Credentials > Add:**

- **Name:** AD Service Account - WinRM
- **Credential Type:** Machine
- **Username:** ansible_svc (*Do not include domain prefix or @YOURDOMAIN.COM suffix here*)
- **Password:** <Service Account Password>
- **Privilege Escalation:** None (*WinRM operates under the authenticated token scope*)

Security Mandate: Never hardcode service account credentials or passwords in playbooks, role defaults, or inventory files. AAP securely injects credentials at runtime.

Step 3: Link Source Control (Project)

Navigate to **Projects > Add:**

- **Name:** Windows Automation Playbooks

- **Execution Environment:** Windows Active Directory EE
- **Source Control Type:** Git
- **Source Control URL:** <https://git.yourdomain.com/automation/windows-ad.git>
- **Source Control Branch/Tag:** main

Step 4: Define Inventory & Group Variables

Navigate to **Inventories > Add > Add Standard Inventory**:

1. **Details:** Name the inventory (e.g., Active Directory Managed Endpoints).
2. **Hosts:** Add target systems using **FQDN only** (e.g., dc01.yourdomain.com).
3. **Variables:** Paste connection settings into the inventory YAML variables editor:

```
---
ansible_connection: winrm
ansible_port: 5986
ansible_winrm_transport: kerberos
ansible_winrm_realm: YOURDOMAIN.COM
ansible_winrm_server_cert_validation: validate
```

Step 5: Construct Job Template

Navigate to **Templates > Add > Add Job Template**:

- **Name:** WinRM Verification & Health Check
- **Job Type:** Run
- **Inventory:** Active Directory Managed Endpoints
- **Project:** Windows Automation Playbooks
- **Execution Environment:** Windows Active Directory EE
- **Playbook:** 01_winrm_ping.yml
- **Credentials:** AD Service Account - WinRM

3.3 Automated Runtime Ticket Execution

1. AAP spins up an isolated container instance using the designated **Execution Environment**.
2. AAP injects the **Machine Credential** into the container context.
3. System Kerberos libraries read /etc/krb5.conf inside the EE and request a Kerberos Ticket-Granting Ticket (TGT) from the Domain Controller.
4. pywinrm establishes an encrypted HTTPS session to port 5986 on the target host using

the active Kerberos ticket.

5. Upon job completion, the temporary container instance and active ticket cache are destroyed.

3.4 Execution Diagnostics & Troubleshooting

UI Error Output	Root Cause	Remediation
Kerberos module not found	Job launched using default EE instead of custom Kerberos EE	Edit Job Template and explicitly select Windows Active Directory EE.
kinit: Cannot find KDC for realm	Lowercase realm or missing DNS record in krb5.conf	Ensure <code>ansible_winrm_realm</code> is ALL CAPS and DNS resolves DC hostnames.
401 Unauthorized	Invalid password or incorrect username formatting	Verify Machine Credential password; ensure username is bare (e.g., <code>ansible_svc</code>).
Connection timed out	Port 5986 blocked or host unreachable	Verify network egress rules from AAP execution nodes to target host on port 5986.

4. Infrastructure Prerequisites

Standard Active Directory, WinRM, and network prerequisites required for remote domain management.

4.1 Infrastructure & Service Requirements

Component	Scope	Technical Requirement
Active Directory	Accounts & SPNs	Standard unprivileged AD service account; target endpoints require registered WinRM Service Principal Names.
Network & Firewall	Port Access	Outbound connectivity on TCP/UDP 88, 464, 389/636, 5986, and 53. Target endpoints require valid A and PTR DNS records.
PKI & Security	Certificates	Base64 PEM-encoded Root/Intermediate CA certificates and unblocked HTTP access to CDP/CRL endpoints.
Container Platform	Registry & Time	Internal container registry repository and NTP clock drift maintained under 5 minutes.

4.2 Network Firewall Matrix

Outbound traffic required from the execution subnet:

- **Port 88 (TCP/UDP):** Active Directory Domain Controllers (Kerberos ticket issuance)
- **Port 464 (TCP/UDP):** Active Directory Domain Controllers (Kerberos password management)
- **Port 389/636 (TCP/UDP):** Active Directory Domain Controllers (LDAP/LDAPS SPN discovery)
- **Port 53 (TCP/UDP):** Internal DNS Servers (Domain and host resolution)
- **Port 5986 (TCP):** Target Windows Endpoints (Encrypted WinRM HTTPS remoting)
- **Port 80/443 (TCP):** Internal PKI Web Servers (Certificate Revocation List validation)

DNS Resolution Mandate: Target endpoints must be referenced by Fully Qualified Domain Name (e.g., win01.yourdomain.com). Kerberos authentication rejects IP-based connections. Internal DNS servers must resolve both forward (A) and reverse (PTR) records.

4.3 Active Directory & Kerberos Standards

- **Case-Sensitive Kerberos Realms:** AD domain names must be specified in **ALL CAPS** across configuration files (e.g., YOURDOMAIN.COM) per Kerberos RFC specifications.
- **Kerberos Time Synchronization:** Clock skew across Domain Controllers, execution nodes, and target endpoints must not exceed 5 minutes.
- **WinRM HTTPS Listeners:** Target endpoints must have active WinRM HTTPS listeners configured on port 5986.

4.4 PKI & Certificate Validation

- **Trusted Certificate Authorities:** Base64 PEM-encoded (.crt/.pem) issuing CA certificates are required for TLS trust verification.
- **Reachable CRL Endpoints:** Certificate Revocation List (CRL) distribution points embedded in target certificates must be reachable over HTTP to prevent connection timeouts.

5. Disconnected Supply Chain & Air-Gapped Build Guide

Procedures for packaging, staging, and building custom Execution Environments in air-gapped or network-isolated enterprise environments.

5.1 Base Container Image Mirroring

Transfer the Red Hat Ansible Automation Platform base image from an internet-connected staging host to your internal enterprise registry using skopeo.

```
# 1. Login to Red Hat Registry and Internal Registry
skopeo login registry.redhat.io
skopeo login my-registry.com
```

```
# 2. Mirror base image directly between registries
```

```
skopeo copy \  
docker://registry.redhat.io/ansible-automation-platform-24/ee-supported-rhel9:latest \  
docker://my-registry.com/automation/ee-supported-rhel9:latest
```

Tarball Transfer Alternative: If direct network transfer between registries is blocked, save the image as a tar archive to offline media:

```
podman pull registry.redhat.io/ansible-automation-platform-24/ee-supported-rhel9:latest  
podman save -o ee-supported-rhel9.tar  
registry.redhat.io/ansible-automation-platform-24/ee-supported-rhel9:latest  
# Import on internal network:  
podman load -i ee-supported-rhel9.tar  
podman tag registry.redhat.io/ansible-automation-platform-24/ee-supported-rhel9:latest  
my-registry.com/automation/ee-supported-rhel9:latest  
podman push my-registry.com/automation/ee-supported-rhel9:latest
```

5.2 Offline Python Wheel Packaging

Download `pywinrm[kerberos]` and associated compiled binaries on an internet-connected host, then stage the wheel files for internal build access.

```
# On Internet-Connected Staging Host:  
mkdir -p ./python-wheels  
pip download \  
--dest ./python-wheels \  
--only-binary=:all: \  
--platform manylinux2014_x86_64 \  
--python-version 3.11 \  
"pywinrm[kerberos]"
```

5.3 Disconnected Build Definition (`execution-environment.yml`)

Configure `execution-environment.yml` to utilize internal container registries, local yum repositories, and offline Python wheel archives.

version: 3

build_arg_defaults:

EE_BASE_IMAGE: 'my-registry.com/automation/ee-supported-rhel9:latest'

dependencies:

ansible_core:

package_pip: ansible-core

system:

- krb5-workstation

- krb5-libs

- ca-certificates

python:

- ./python-wheels/pywinrm-*.whl

- ./python-wheels/requests_kerberos-*.whl

additional_build_steps:

prepend_base:

- COPY internal-root-ca.crt /etc/pki/ca-trust/source/anchors/internal-root-ca.crt

- RUN update-ca-trust

- COPY ./python-wheels /tmp/python-wheels

append_final:

- COPY krb5.conf /etc/krb5.conf

- RUN rm -rf /tmp/python-wheels

5.4 Air-Gapped Network Diagnostics

Execute these CLI diagnostics from the execution node prior to running playbooks to confirm network pathways and internal repository access.

1. Test DNS FQDN resolution for Domain Controllers

dig +short dc01.yourdomain.com

2. Test reverse PTR record lookup

dig +short -x <DC_IP_ADDRESS>

3. Test egress connectivity to WinRM HTTPS port
nc -zvw3 win01.yourdomain.com 5986

4. Test Kerberos port egress to Domain Controller
nc -zvw3 dc01.yourdomain.com 88

5. Validate internal HTTP CRL distribution endpoint
curl -I http://crl.yourdomain.com/pki/internal-ca.crl